CS559: COMPUTER SYSTEMS DESIGN

FAULT TOLERANCE

Dr. Gagan Raj Gupta



MOTIVATION

- Three fundamental scalability patterns:
  - functional decomposition, data partitioning, and replication.
- They all have one thing in common: they increase the number of moving parts (machines, services, processes, etc.) in our applications.
- But since every part has a probability of failing, the more moving parts there are, the higher the chance that any of them will fail.
- Eventually, **anything that can go wrong will go wrong**, power outages, hardware faults, software crashes, memory leaks— you name it.



MEASURES OF RELIABILITY

- **Availability and Mean Time to Failure**
- Time to repair (TTR). Time to failure (TTF).
- the *mean time to failure* (MTTF) and the *mean time to repair* (MTTR)

$$MTTF = \frac{1}{N}\sum_{i=1}^{N} TTF_i \qquad MTTR = \frac{1}{N}\sum_{i=1}^{N} TTR_i$$

$$Availability = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF + MTTR} = \frac{MTBF - MTTR}{MTBF}$$

$$\text{Down time} = (1 - Availability) = \frac{MTTR}{MTBF}$$

$$Availability = \frac{\text{time system was running}}{\text{time system should have been running}} = \frac{\sum_{i=1}^{N} TTF_i}{\sum_{i=1}^{N} (TTF_i + TTR_i)}$$



CONDITIONAL FAILURE RATE

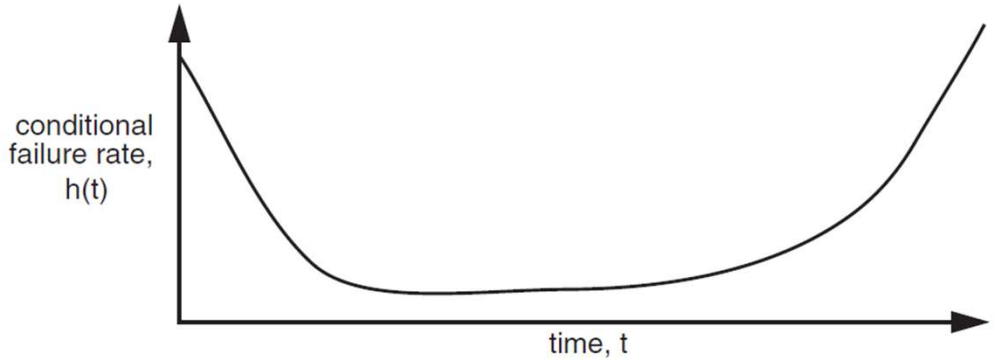


FIGURE 8.1

A bathtub curve, showing how the conditional failure rate of a component changes with time.

conditional failure rate *h(t)* of a module, defined to be the probability that the module fails between time *t* and time *t + dt,* given that the component is still working at time *t*.

- Initially high, i.e. some of them fail very soon
- If they don't fail early, they tend to have a longer life
- Eventually, failure rate becomes quite high



RELIABILITY FUNCTIONS

The *reliability*, *R*, of a module is defined to be

$$R(t) = Pr\left(\begin{array}{c}\text{the module has not yet failed at time t, given that}\\ \text{the module was operating at time 0}\end{array}\right) \qquad \text{Eq. 8–5}$$

and the unconditional failure rate *f(t)* is defined to be

$$f(t) = Pr(\text{module fails between t and t + dt}) \qquad \text{Eq. 8–6}$$

(The bathtub curve and these two reliability functions are three ways of presenting the same information. If you are rusty on probability, a brief reminder of how they are related appears in Sidebar 8.1.) Once *f(t)* is at hand, one can directly calculate the MTTF:

$$MTTF = \int_0^\infty t \cdot f(t)dt \qquad \text{Eq. 8–7}$$



AVAILABILITY

- To guarantee just two nines, an application can only be unavailable for up to 15 minutes a day. That's very little time to take any manual action when something goes wrong.
- And if we strive for three nines, we only have 43 minutes per *month* available.
- Clearly, the more nines we want, the faster our systems need to detect, react to, and repair faults as they occur.
- In this part, we will discuss a variety of resiliency best practices and patterns to achieve that.
- **Systems that continue to provide partial service in the face of failure are called *fail-soft***



FAULTS, ERRORS AND FAILURES

F*ault:* an underlying defect, imperfection, or flaw that has the potential to cause problems, whether it actually has, has not, or ever will.

Error: When a fault is active, **wrong results appear in data values or control signals**. These wrong results are *errors.*

Failure: Not producing the intended result at an interface is the formal definition of a *failure*. If an error is not detected and masked, the module probably does not perform to its specification.



FAULT OR FAILURE?

The distinction between fault and failure is closely tied to modularity and the building of systems out of well-defined subsystems.

In a system built of subsystems, the failure of a subsystem is a fault from the point of view of the larger subsystem that contains it.

That fault may cause an error that leads to the failure of the larger subsystem, unless the larger subsystem anticipates the possibility of the first one failing, detects the resulting error, and masks it.

Fault tolerance thus consists of noticing active faults and component subsystem failures and doing something helpful in response.



EXAMPLES OF FAULTS

- **Software fault:** A programming mistake, such as placing a less-than sign where there should be a less-than-or-equal sign. This fault may never have caused any trouble because the combination of events that requires the equality case to be handled correctly has not yet occurred. Or, perhaps it is the reason that the system crashes twice a day. If so, those crashes are failures.
- **Hardware fault:** A gate whose output is stuck at the value ZERO. Until something depends on the gate correctly producing the output value ONE, nothing goes wrong. If you publish a paper with an incorrect sum that was calculated by this gate, a failure has occurred. Furthermore, the paper now contains a fault that may lead some reader to do something that causes a failure elsewhere.
- **Design fault:** A miscalculation that has led to installing too little memory in a telephone switch. It may be months or years until the first time that the presented load is great enough that the switch actually begins failing to accept calls that its specification says it should be able to handle.



EXAMPLES OF FAULTS

- **Implementation fault:** Installing less memory than the design called for. In this case the failure may be identical to the one in the previous example of a design fault, but the fault itself is different.
- **Operations fault:** The operator responsible for running the weekly payroll ran the payroll program twice last Friday. Even though the operator shredded the extra checks, this fault has probably filled the payroll database with errors such as wrong values for year-to-date tax payments.
- **Environment fault:** Lightning strikes a power line, causing a voltage surge. The computer is still running, but a register that was being updated at that instant now has several bits in error. Environment faults come in all sizes, from bacteria contaminating ink-jet printer cartridges to a storm surge washing an entire building out to sea.



TRANSIENT VS PERSISTENT FAULTS

- A *transient* fault, also known as a *single-event upset*, is temporary, triggered by some passing external event such as lightning striking a power line or a cosmic ray passing through a chip.
- It is usually possible to mask an error caused by a transient fault by trying the operation again.
- An error that is successfully masked by retry is known as a *soft error*.
- A *persistent* fault continues to produce errors, no matter how many times one retries, and the corresponding errors are called *hard errors*.



Sidebar 8.4: Detecting failures with heartbeats. An activity such as a Web server is usually intended to keep running indefinitely. If it fails (perhaps by crashing) its clients may notice that it has stopped responding, but clients are not typically in a position to restart the server. Something more systematic is needed to detect the failure and initiate recovery. One helpful technique is to program the thread that should be performing the activity to send a periodic signal to another thread (or a message to a monitoring service) that says, in effect, "I'm still OK". The periodic signal is known as a *heartbeat* and the observing thread or service is known as a *watchdog*.

The watchdog service sets a timer, and on receipt of a heartbeat message it restarts the timer. If the timer ever expires, the watchdog assumes that the monitored service has gotten into trouble and it initiates recovery. One limitation of this technique is that if the monitored service fails in such a way that the only thing it does is send heartbeat signals, the failure will go undetected.

As with all fixed timers, choosing a good heartbeat interval is an engineering challenge. Setting the interval too short wastes resources sending and responding to heartbeat signals. Setting the interval too long delays detection of failures. Since detection is a prerequisite to repair, a long heartbeat interval increases MTTR and thus reduces availability.



BUILDING RELIABLE SYSTEMS

- ***Error detection*:** discovering that there is an error in a data value or control signal. Error detection is accomplished with the help of *redundancy*, extra information that can verify correctness.
- ***Error containment*:** limiting how far the effects of an error propagate. Error containment comes from careful application of modularity. When discussing reliability, a *module* is usually taken to be the unit that fails independently of other such units. It is also usually the unit of repair and replacement.
- ***Error masking*:** ensuring correct operation despite the error. Error masking is accomplished by providing enough additional redundancy that it is possible to discover correct, or at least acceptably close, values of the erroneous data or control signal. When masking involves changing incorrect values to correct ones, it is usually called *error correction*.



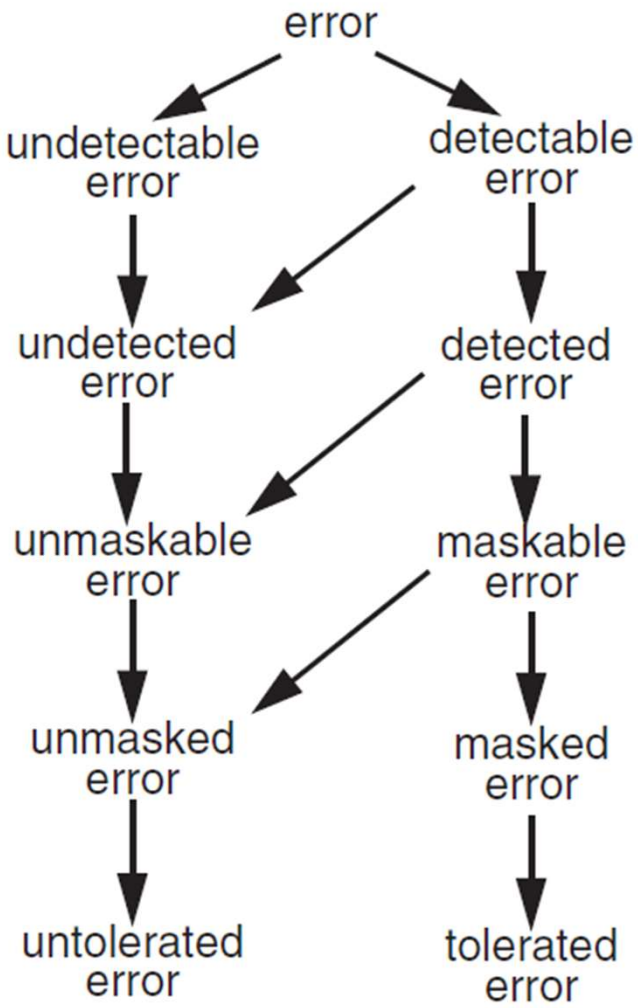
ERROR CONTAINMENT

- Mask the error, so the higher-level subsystem does not realize that anything went wrong.
- Detect and report the error at its interface, producing what is called a *fail-fast* design. Fail-fast subsystems simplify the job of detection and masking for the next higher-level subsystem.
- Immediately stop dead, thereby hoping to limit propagation of bad values, a technique known as *fail-stop*. Fail-stop subsystems require that the higher-level subsystem take some additional measure to discover the failure, for example by setting a timer and responding to its expiration.
- Do nothing, simply failing without warning. At the interface, the error may have contaminated any or all output values. (Informally called a “crash” or perhaps “failthud”.)



MASKING ERRORS

- A *detectable error* is one that can be detected reliably. If a detection procedure is in place and the error occurs, the system discovers it with near certainty and it becomes a *detected* error
- A *maskable error* is one for which it is possible to devise a procedure to recover correctness. **If a masking procedure is in place and the error occurs, is detected, and is masked, the error is said to be *tolerated*.**
- Conversely, an *untolerated* error is one that is undetectable, undetected, unmaskable, or unmasked



FAULT TOLERANT DESIGN PROCESS

- One way to design a reliable system would be to build it entirely of components that are individually so reliable that their chance of failure can be neglected. This technique is known as *fault avoidance*.
- If all *N* of the components of a system must work, the probability of any one component failing is *p*, and component failures are independent of one another, then the probability that the system works is $(1 - p)^N$.
- No matter how small *p* may be, there is some value of *N* beyond which this probability becomes too small for the system to be useful



- Begin to develop a fault-tolerance model,
  - Identify every potential fault.
  - Estimate the risk of each fault.
- Where the risk is too high, design methods to detect the resulting errors.
- Apply modularity to contain the damage from the high-risk errors.
- Design and implement procedures that can mask the detected errors:
  - Temporal redundancy. Retry the operation, using the same components.
  - Spatial redundancy. Have different components do the operation.
- Update the fault-tolerance model to account for those improvements.
- Iterate the design and the model until the probability of untolerated faults is low enough that it is acceptable.
- Observe the system in the field: Check logs of how many errors the system is successfully masking.
- Perform postmortems on failures and identify *all* of the reasons for each failure. Use the logs of masked faults and the postmortem reports about failures to revise and improve the fault-tolerance model and reiterate the design.



SAFETY NET APPROACH

- **Be explicit :** *Get all of the assumptions out on the table.*
- *Design for iteration.* It is difficult or impossible to anticipate all of the ways that things can go wrong
  - When working with a fast-changing technology it can be hard to estimate probabilities of failure in components and in their organization, a fault tolerant design must include feedback about actual error rates, evaluation of that feedback, and update of the design as field experience is gained. To act on the feedback requires having a fault tolerance model that is explicit about reliability assumptions
- S*afety margin principle*, When fault tolerant systems fail, it is usually not because they had inadequate fault tolerance, but because the number of failures grew unnoticed until the fault tolerance of the design was exceeded. The key requirement is that the system **log all failures** and that someone pay attention to the logs.
- *Keep digging.* Complex systems fail for complex reasons. When a failure of a system that is supposed to be reliable does occur, always look beyond the first, obvious cause. It is nearly always the case that there are actually several contributing causes and that there was something about the mind set of the designer that allowed each of those causes to creep in to the design.
- *Adopt sweeping simplifications*.



FAULT TOLERANT SOFTWARE DESIGN

- Designers of reliable systems usually assume that, despite the best efforts of programmers there will always be a residue of software faults, just as there is also always a residue of hardware, operation, and environment faults.
- Current state of a running program tends to be widely distributed. Parts of that state may be in non-volatile storage, while other parts are in temporary variables held in volatile memory locations, processor registers, and kernel tables. This wide distribution of state makes containment of errors problematic.
- Design approach: Systematically divide the current state of a running program into two mutually exclusive categories and separate the two categories with a firewall. The two categories are:
  - State that the system can safely abandon in the event of a failure.
  - State whose integrity the system should preserve despite failure.



EXAMPLE OF SWEEPING SIMPLIFICATION

- Upon detecting a failure, the plan becomes to abandon all state in the first category and instead concentrate just on maintaining the integrity of the data in the second category. An important part of the strategy is an important *sweeping simplification*: classify the state of running threads (that is, the thread table, stacks, and registers) as abandon-able.
- When a failure occurs, the system abandons the thread or threads that were running at the time and instead expects a restart procedure, the system operator, or the individual user to start a new set of threads with a clean slate.
- The new thread or threads can then, working with only the data found in the second category, verify the integrity of that data and return to normal operation. The primary challenge then becomes to build a firewall that can protect the integrity of the second category of data despite the failure.
- The goal can be succinctly stated: to detect failures and stop the system before it reaches the next PUT operation (to non-volatile storage, disk for eg), thus making the volatile storage (memory) medium the error containment boundary.



DATA SYSTEMS DESIGN PRINCIPLES

- Prepare for failure by recognizing that all state in volatile memory devices can vanish at any instant, without warning. When it does vanish, automatically launch new threads that start by restoring the data in non-volatile storage to a consistent, easily described state. The techniques to do this restoration are called *recovery*. Doing recovery systematically involves **Atomicity.**
- Protect the data in non-volatile storage using replication, thus creating the class of storage known as ***durable* storage.** Replicating data can be a straightforward application of redundancy. However, there are more effective designs that make use of atomicity and geographical separation of replicas.



DNS FAULT TOLERANT DESIGN

- Independent software implementations
  - Multiple software teams developing different implementations of DNS
  - On different OS, for example
- Firewall between categories of data
- Geographical separation of its replicas

